

GPUMODE Lecture Study Notes: Lecture 69

Quartet 4-Bit Training

Core Problem the Lecture Addresses

Modern large language models are trained using floating-point representations such as FP32, BF16, FP16, and FP8. These formats are computationally expensive but provide large dynamic range and numerical stability.

The lecture investigates a fundamental question:

If language models appear to store only a few bits of information per parameter, why do we need high-precision representations throughout training?

The Quartet project studies whether native FP4 training can achieve better compute-quality trade-offs than FP8 or BF16 training.

The goal is not simply:

- Can FP4 train?

but rather:

- Under a fixed training budget, is FP4 optimal?
- Under a fixed inference budget, is FP4 optimal?
- Can lower precision permit larger models that outperform smaller high-precision models?

The work introduces:

1. A scaling-law framework for quantized training.
2. A budget-normalized comparison methodology.
3. New FP4 forward and backward quantization schemes.
4. High-performance Blackwell kernels.

Required Background

Gradient-Based Learning

Model parameters are optimized through gradient descent.

Given parameters

$$\theta$$

and loss

$$L(\theta),$$

training performs updates

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L.$$

This requires differentiable numerical representations.

Floating Point Numbers

A floating-point number consists of:

$$\text{Value} = (-1)^s \times m \times 2^e,$$

where:

- s = sign bit
- m = mantissa
- e = exponent

Floating point provides:

- large dynamic range
- approximate closure under arithmetic
- stable gradient propagation

Information Stored Per Parameter

Empirical studies suggest that large language models effectively store roughly

1–2

bits of useful information per parameter.

This creates a mismatch:

Optimization Precision $\gg$ Information Content.

This motivates low-bit training.

Main Concepts

Training Precision Versus Capacity

High precision provides:

- optimization stability
- numerical accuracy
- large dynamic range

However:

- model knowledge storage is far lower
- training may be over-provisioned numerically

The challenge is determining how much precision is actually necessary.

Low Precision Data Types

Modern hardware supports:

- FP32
- BF16
- FP16
- FP8
- FP4
- INT8
- other specialized formats

These formats are not merely storage formats.

Hardware provides:

- Tensor Core support
- matrix multiplication instructions
- specialized accumulation paths

Thus lower precision can directly accelerate training.

Accuracy-Speed Trade-Off

Reducing precision provides:

Higher Speed

but potentially causes

Lower Accuracy.

Historically many papers demonstrated:

- near-baseline quality
- measurable speedups

Examples discussed include:

- Jetfire
- Halo
- Logarithmic Bias Quantization
- INT8 Transformer Training

The lecture argues that comparing isolated training runs is insufficient.

Scaling-Law Based Comparison

Traditional comparisons answer:

Does method A outperform method B on one experiment?

The authors instead ask:

Which method is optimal across an entire family of models?

Classical LLM Scaling Laws

A scaling law predicts loss from:

$$N = \text{parameter count}$$

and

$$D = \text{training tokens.}$$

Generally:

$$L = f(N, D).$$

This enables extrapolation beyond trained models.

Quantized Training Scaling Law

Quartet extends scaling laws by introducing:

$$P_f = \text{forward precision}$$

and

$$P_b = \text{backward precision.}$$

The intuition is:

- Forward quantization reduces effective model capacity.
- Backward quantization slows optimization.

Forward quantization behaves as:

$$N_{\text{effective}} = \alpha(P_f)N.$$

where

$$0 < \alpha(P_f) \leq 1.$$

Backward quantization behaves as:

$$D_{\text{effective}} = \beta(P_b)D.$$

where

$$0 < \beta(P_b) \leq 1.$$

Therefore:

$$L = f(\alpha(P_f)N, \beta(P_b)D).$$

Interpretation

Forward quantization:

- changes representational capacity
- behaves like reducing model size

Backward quantization:

- injects optimization noise
- slows convergence
- behaves like reducing token count

Budget-Normalized Comparison

The major contribution is introducing budget-normalized evaluation.

Inference Budget

Inference cost scales approximately as:

$$C_{\text{infer}} \propto N.$$

If FP4 is twice as fast:

$$S_f = 2,$$

then under equal inference cost:

$$N_{\text{FP4}} = 2N_{\text{FP8}}.$$

Thus lower precision can buy larger models.

Training Budget

Training cost scales approximately as:

$$C_{\text{train}} \propto ND.$$

Training speedups depend on both:

- forward speedup
- backward speedup

This permits compute-normalized comparisons.

Optimality Regions

The authors fit scaling laws and then determine:

$$\arg \min L$$

for each compute budget.

This produces optimality maps indicating where:

- FP8 is optimal
- FP4 forward-only is optimal
- FP4 backward-only is optimal
- full FP4 is optimal

Many modern LLMs fall inside FP4-optimal regions.

Quartet Quantization Method

Forward Pass

The best forward quantization scheme was Quest.

Quest uses:

- standard deviation scaling
- Hadamard transforms
- clipping
- quadratic-error minimization

The Hadamard transform approximately Gaussianizes activations.

This improves quantization efficiency.

Backward Pass

The best backward method used:

- stochastic rounding
- Hadamard transforms
- unbiased gradient estimation

The goal is maintaining

$$\mathbb{E}[\hat{g}] = g.$$

Unbiased gradients preserve convergence properties.

Hardware-Level Explanation

MXFP4 Format

Quartet targets Blackwell MXFP4.

Each value uses:

$$E2M1$$

representation.

Bit allocation:

$$1 + 2 + 1 = 4$$

bits.

Specifically:

- 1 sign bit
- 2 exponent bits
- 1 mantissa bit

Microscaling

Groups of 32 values share a scale.

The scale is represented using:

$$E8M0$$

FP8 metadata.

Therefore:

- 32 FP4 values
- 1 shared scale

form a quantization block.

Tensor Core Execution

Blackwell Tensor Cores directly consume:

- FP4 values
- scaling metadata

during GEMM execution.

Conceptually:

$$C = (A, s_A) \times (B, s_B).$$

Why GEMMs Matter

Transformer runtime is dominated by:

GEMM.

Thus optimizing matrix multiplication determines overall performance.

Hadamard Transform Analysis

Small Hadamard Sizes

For size

$$H < 256,$$

the operation is memory bound.

Performance depends primarily on memory movement.

Large Hadamard Sizes

For

$$H \geq 256,$$

the operation becomes compute bound.

Fast Walsh-Hadamard Transform complexity:

$$O(n \log n).$$

Key Insight

Because small Hadamard transforms are memory bound:

- they can be hidden behind memory loads
- they introduce little additional overhead

This makes Hadamard normalization practical.

Kernel Design

Fused Quantization Kernel

Input:

- BF16 activations
- BF16 weights

Pipeline:

1. Hadamard transform
2. FP32 accumulation
3. FP4 quantization
4. scale generation
5. clipping-mask generation

Outputs:

- FP4 values
- FP8 scales
- clipping mask

Custom Epilogue

The epilogue computes quantization outputs.

Advantages:

- thread-local computation
- no synchronization
- nearly free overhead

Metadata Rearrangement

Blackwell requires metadata in a specific layout.

A rearrangement kernel converts generated scales into Tensor Core format.

This currently remains separate but could be fused in future work.

Performance Results

Observed speedups:

Versus FP8

Forward:

$$\approx 2.2\times$$

Backward:

$$\approx 1.5\times$$

Reason for Gap

Forward kernels received more optimization effort.

Backward kernels remain active work.

The expectation is that future kernels will improve backward speedups.

Mathematical Reasoning

Forward Quantization Model

Capacity reduction:

$$N \rightarrow \alpha N.$$

Backward Quantization Model

Optimization slowdown:

$$D \rightarrow \beta D.$$

Effective Scaling Law

Conceptually:

$$L = L_{\infty} + A(\alpha N)^{-\gamma} + B(\beta D)^{-\delta}.$$

This preserves classical scaling-law structure while incorporating quantization.

Code Walkthrough

Representative pipeline:

```
Input BF16 Matrix
|
Hadamard Transform
|
FP32 Accumulation
|
Quest Quantization
|
Generate FP8 Scales
|
Generate Clip Mask
|
MXFP4 GEMM
|
Output
```

Hardware interpretation:

- Tensor Cores execute MXFP4 GEMM.
- Shared memory stores intermediate outputs.
- Hadamard transform normalizes distributions.
- Quantization metadata feeds Tensor Core instructions.

Performance Intuition

Why FP4 can win:

1. Lower precision increases arithmetic throughput.
2. Lower precision reduces memory traffic.
3. Larger models become affordable.
4. Scaling laws predict quality gains from increased model size.

Therefore:

Larger FP4 Model > Smaller FP8 Model

under equal compute budgets.

This is the central argument of the paper.

Common Misconceptions

FP4 Must Match FP8 Accuracy

False.

The relevant comparison is:

Accuracy at Equal Compute.

Post-Training Quantization and Native Training Are Equivalent

False.

Post-training quantization perturbs a trained solution.

Native FP4 training learns directly within the quantized space.

Backward Quantization Changes Final Optimum

Not necessarily.

The lecture argues that backward quantization primarily slows convergence rather than changing the eventual optimum.

Practical Takeaways

- FP4 training is increasingly realistic on Blackwell GPUs.
- Scaling laws provide a principled way to compare quantization methods.
- Forward quantization mainly affects capacity.

- Backward quantization mainly affects convergence speed.
- Kernel efficiency is as important as quantization quality.
- Hadamard transforms remain a powerful quantization primitive.
- Compute-normalized comparisons are more meaningful than isolated benchmarks.

Project or Experiment Ideas

1. Reproduce scaling-law fitting for FP8 and FP4.
2. Implement Quest quantization in Triton.
3. Benchmark Hadamard sizes 32, 64, 128, and 256.
4. Compare stochastic versus deterministic rounding.
5. Measure convergence slowdown under backward quantization.
6. Profile MXFP4 kernels using Nsight Compute.
7. Compare MXFP4 and NVFP4 on Blackwell GPUs.
8. Explore larger microscaling group sizes.

Final Mental Model

Quartet reframes low-precision training as a resource-allocation problem rather than a numerical-accuracy problem. FP4 reduces representational precision, but the resulting speedups allow larger models and more efficient compute utilization. By combining scaling laws, compute normalization, Hadamard-based quantization, stochastic gradient estimation, and optimized Blackwell kernels, the authors show that native FP4 training can become the optimal operating point for many realistic large language model training regimes.